Chapter 12

Regex Evaluator Using NFA and DFA

Konduru Navadeep (251132), Alavala Prathap Reddy (251107)

CS3104 Mini Project Final Report

Contents

12 Regex Evaluator Using NFA and DFA **1**

12.1 Introduction and Problem Statement . . . 4
12.2 Scope and Feature Set . . . 4
12.3 Object-Oriented Architecture . . . 4
12.3.1 Class Design and Associations . . . 5
12.3.2 OOP Principles . . . 5
12.3.3 Abstract Base Class and Virtual Function . . . 5
12.4 System Architecture and Design . . . 6
12.4.1 Use Case Diagram . . . 6
12.4.2 Activity Diagram . . . 7
12.4.3 Sequence Diagram . . . 8
12.5 Core Algorithms . . . 8
12.5.1 Recursive-Descent Parsing . . . 8
12.5.2 Thompson's NFA Construction . . . 8
12.5.3 NFA Operations and Subset Construction . . . 9
12.5.4 DFA Simulation . . . 9
12.6 Selected Code Snippets . . . 9
12.6.1 DFA Matching . . . 9
12.6.2 NFA-to-DFA State Reuse . . . 9
12.7 Formal Specification of the Supported Language . . . 10
12.7.1 Supported operators and examples . . . 10
12.7.2 Operator precedence and parser grammar . . . 10
12.7.3 Recursive-descent parser responsibilities . . . 11
12.8 Detailed Object Model . . . 11
12.8.1 Responsibilities of the main classes . . . 11
12.8.2 Expanded class relationship diagram . . . 11
12.8.3 Object-oriented design discussion . . . 13
12.9 Finite Automata: Visual Construction Walkthrough . . . 14
12.9.1 NFA fragment for a literal . . . 14
12.9.2 NFA fragment for union . . . 14
12.9.3 NFA fragment for Kleene star . . . 14
12.9.4 How epsilon-closure and move work . . . 15
12.9.5 Subset construction: NFA states become DFA states . . . 15
12.10 Worked Example: Evaluating `(a|b)*abb` . . . 16
12.10.1 Example input accepted . . . 16
12.10.2 Example input rejected . . . 16
12.10.3 DFA simulation trace concept . . . 17
12.11 Algorithm Pseudocode . . . 17

12.11.1 Recursive-descent parsing . . . 17
12.11.2 Thompson construction . . . 18
12.11.3 DFA matching . . . 18
12.12 Complexity and Resource Considerations . . . 18
12.13 Complete Test Coverage Summary . . . 19
12.14 Diagram-Based Demonstration Guide . . . 19
12.14.1 TikZ Result-State Illustration . . . 20
12.15 Implementation and Usage . . . 20
12.16 Results and Testing . . . 21
12.17 Limitations and Future Enhancements . . . 21
12.18 Conclusion . . . 21

12.1 Introduction and Problem Statement

Regular expressions provide a compact way to describe patterns in strings. Checking whether a string follows a regular expression is a common task in text processing, input validation, lexical analysis, and compiler design. A full-featured production regex library can be complex and may hide the algorithms used to perform matching.

This project implements a lightweight regular-expression evaluator in C++. The user supplies a regular expression and a test string. The program parses the expression into a syntax tree, builds a Non-deterministic Finite Automaton (NFA), converts the NFA into a Deterministic Finite Automaton (DFA), and uses the DFA to decide whether the entire input string is accepted. The result is printed as `ACCEPT` or `REJECT`.

The main problem addressed is to demonstrate the internal stages of regex evaluation using object-oriented programming (OOP), finite-automata algorithms, and the C++ Standard Template Library (STL), rather than relying on a built-in regex library.

12.2 Scope and Feature Set

The implementation focuses on a small and understandable subset of regular-expression syntax.

- **Literal characters:** matches a character such as `a`.
- **Concatenation:** matches expressions in sequence, for example `abc`.
- **Union:** the `|` operator represents alternatives, for example `a|b`.
- **Kleene star:** the `*` operator represents zero or more repetitions, for example `a*`.
- **Grouping:** parentheses group expressions, for example `(a|b)*`.
- **Automata construction:** builds an NFA using Thompson's construction and converts it to a DFA using subset construction.
- **Input validation:** rejects malformed expressions handled by the parser, including unmatched parentheses, a trailing union operator, a standalone star, repeated stars, and empty groups.
- **Automated tests:** a PowerShell script compiles the C++ source files and checks 21 input cases against expected results.

The current implementation is limited. Operators such as `+`, `?`, character classes, escape sequences, and wildcard matching are outside its current scope.

12.3 Object-Oriented Architecture

The program separates parsing, syntax-tree representation, NFA construction, NFA operations, DFA construction, and simulation into individual classes. This separation makes the implementation easier to understand, test, and extend.

12.3.1 Class Design and Associations

`RegexNode` is the abstract base class for syntax-tree nodes. `LiteralNode`, `ConcatNode`, `UnionNode`, and `StarNode` derive from it. The parser produces a tree of these nodes. The NFA builder traverses the tree and creates NFA fragments. NFA and DFA state classes store transitions, while simulator classes perform matching.

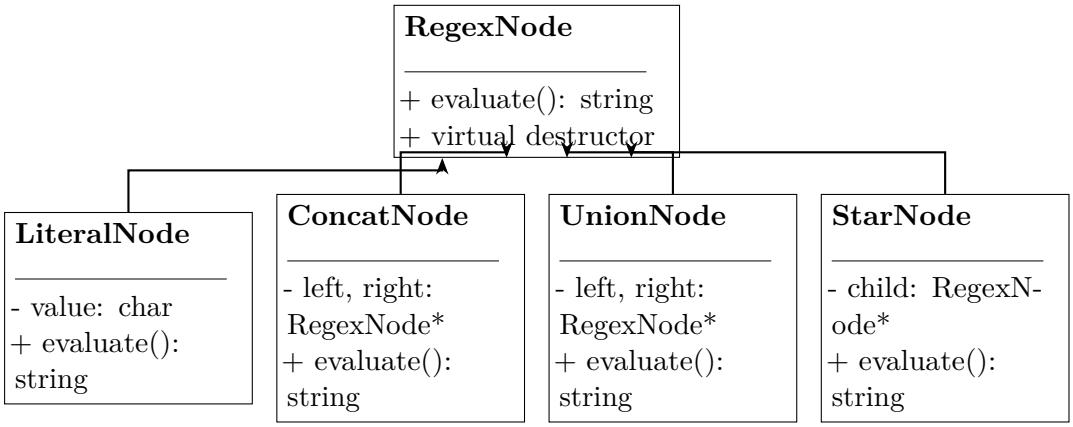


Figure 12.1: Simplified class diagram for the syntax-tree node hierarchy.

12.3.2 OOP Principles

- **Abstraction:** `RegexNode` defines a common interface through the pure virtual function `evaluate()`.
- **Inheritance:** the literal, concatenation, union, and star nodes inherit from `RegexNode`.
- **Polymorphism:** derived node implementations provide their own `evaluate()` behaviour and can be handled through a base-class pointer.
- **Encapsulation:** node data and automaton state/transition information are maintained within their respective classes and exposed through class interfaces.
- **Modularity:** parser, builders, operations, and simulators have separate responsibilities.

12.3.3 Abstract Base Class and Virtual Function

The following excerpt shows the common interface used by syntax-tree nodes.

```
class RegexNode {
public:
    virtual ~RegexNode() = default;
    virtual std::string evaluate() = 0;
};
```

The assignment of `= 0` makes `evaluate()` a pure virtual function, so `RegexNode` is abstract and cannot be instantiated directly. The virtual destructor allows derived objects to be safely destroyed through a base-class pointer.

12.4 System Architecture and Design

The processing pipeline consists of the following stages:

1. Read the regular expression and input string from the console.
2. Parse the expression using recursive-descent parsing and construct a syntax tree.
3. Convert the syntax tree into an NFA using Thompson's construction.
4. Use epsilon-closure and move operations to determine reachable NFA states.
5. Convert the NFA into a DFA through subset construction.
6. Simulate the DFA on the input string and print the final result.

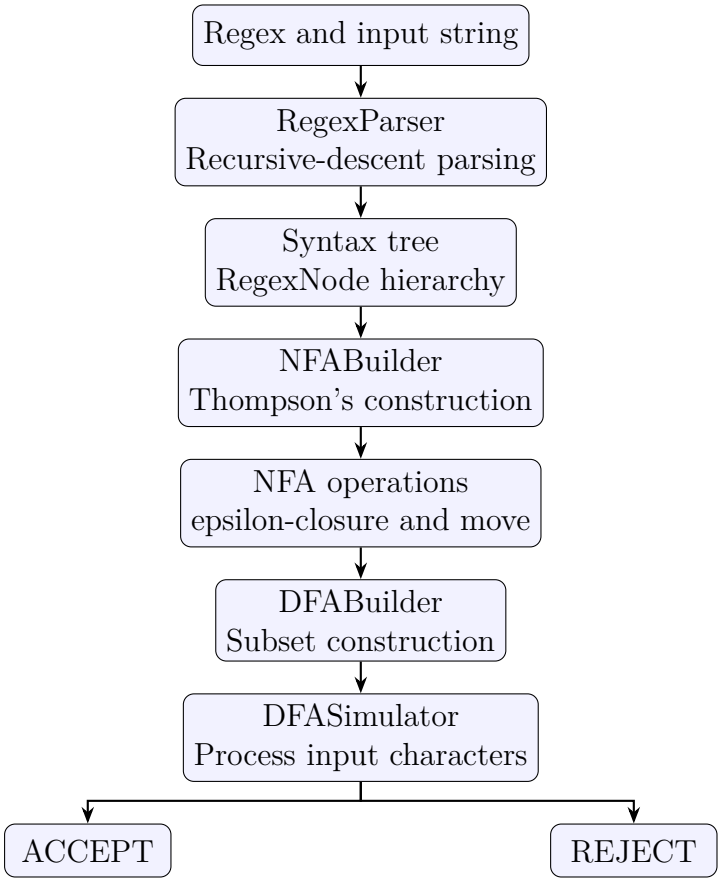


Figure 12.2: High-level architecture of the Regex Evaluator.

12.4.1 Use Case Diagram

The primary actor is the user, who supplies a regular expression and a string to test. The system validates/parses the expression, evaluates the string, and displays the result.

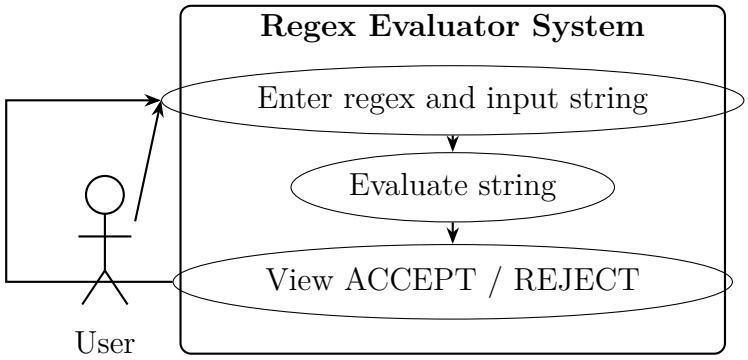


Figure 12.3: Use case diagram.

12.4.2 Activity Diagram

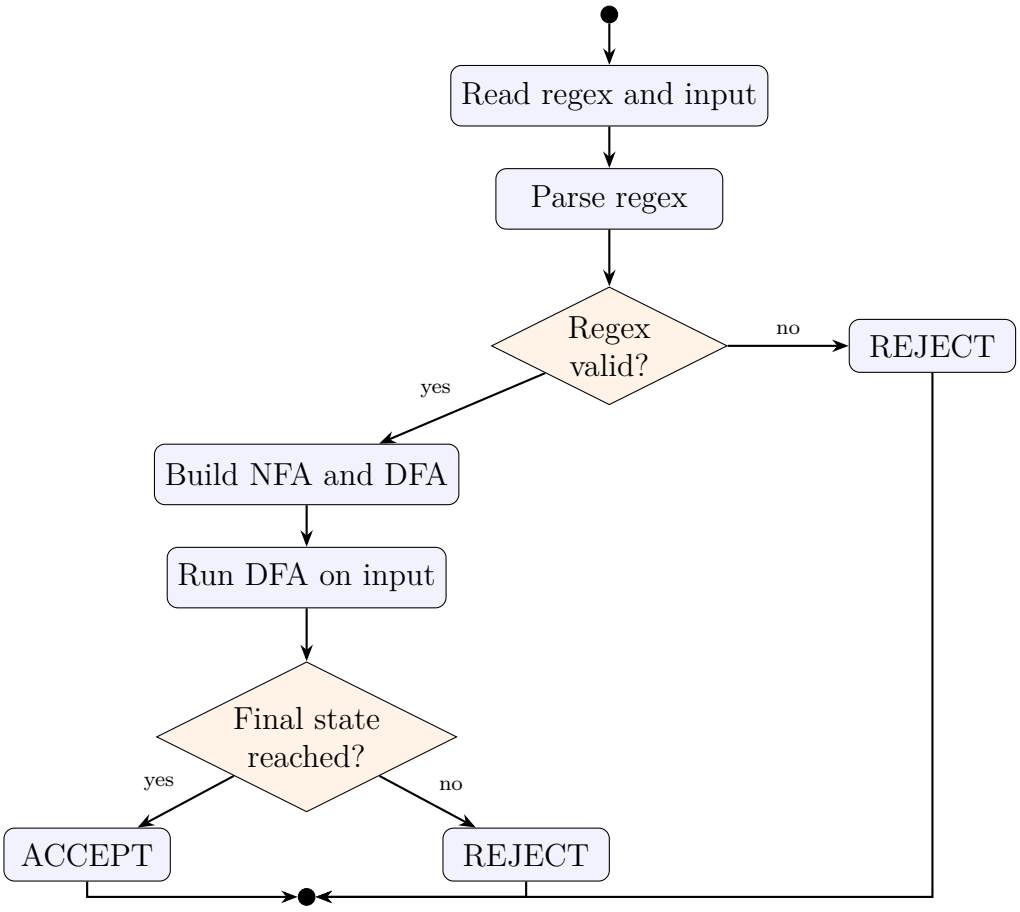


Figure 12.4: Activity diagram for evaluating a regular expression.

12.4.3 Sequence Diagram

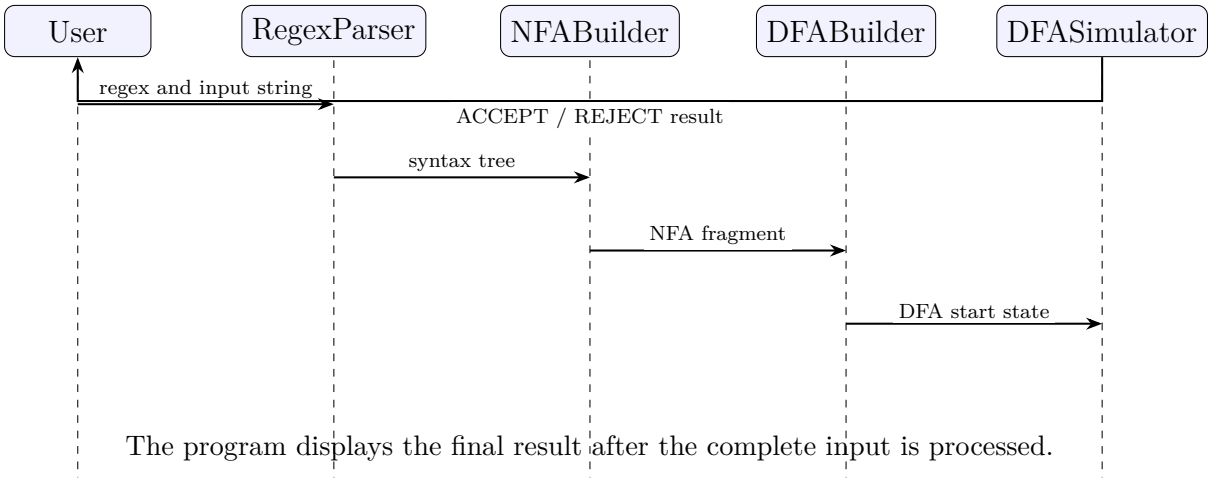


Figure 12.5: Simplified sequence of interactions between the user and core components.

12.5 Core Algorithms

12.5.1 Recursive-Descent Parsing

The parser processes the expression using a set of mutually cooperating functions. `parseExpression()` handles union, `parseTerm()` handles concatenation, `parseFactor()` handles the Kleene-star postfix operator, and `parsePrimary()` handles literals and parenthesized expressions. This structure captures the precedence of the supported operators: grouping and star bind more tightly than concatenation, which binds more tightly than union.

12.5.2 Thompson's NFA Construction

Thompson's construction creates an NFA fragment with a start state and an accepting state for each syntax-tree node.

- For a literal, create two states and connect them using the literal character.
- For concatenation, connect the first fragment's accepting state to the second fragment's start using an epsilon transition.
- For union, create a new start and end state and use epsilon transitions to branch to either alternative and rejoin.
- For Kleene star, add epsilon transitions that allow the child fragment to be skipped or repeated.

In the implementation, the null character `'\0'` represents an epsilon transition.

12.5.3 NFA Operations and Subset Construction

The epsilon-closure operation finds all states reachable using only epsilon transitions. The move operation finds states reachable on a given input symbol. During subset construction, each DFA state represents a set of NFA states. For each symbol in the NFA alphabet, the builder computes move followed by epsilon-closure. A DFA state is accepting when its represented set contains at least one accepting NFA state.

12.5.4 DFA Simulation

The DFA simulator starts from the DFA start state and processes the input from left to right. For each character, it follows the corresponding transition. If a required transition is absent, the string is rejected. After the entire string is consumed, the string is accepted only if the current DFA state is final. This performs full-string matching rather than searching for a substring.

12.6 Selected Code Snippets

12.6.1 DFA Matching

The following excerpt illustrates the deterministic matching stage.

```
bool DFASimulator::matches(DFAState* start,
                           const std::string& input) {
    DFAState* current = start;

    for (char symbol : input) {
        if (current->transitions.count(symbol) == 0)
            return false;
        current = current->transitions[symbol];
    }

    return current->isFinal;
}
```

The transition lookup is deterministic: for each input symbol there is at most one next state. A missing transition immediately rejects the string, while the final-state check determines the result after all characters have been processed.

12.6.2 NFA-to-DFA State Reuse

Subset construction avoids creating duplicate DFA states by checking whether a state already represents the same set of NFA states.

```
DFAState* DFABuilder::findState(
    const std::vector<DFAState*>& states,
    const std::set<NFAState*>& nfaStates) {
    for (DFAState* state : states) {
        if (state->nfaStates == nfaStates)
            return state;
    }
```

```
8     return nullptr;
9 }
```

This check is important because the same set of NFA states may be reached by different input paths. Reusing the existing DFA state prevents duplicate states from being created.

12.7 Formal Specification of the Supported Language

This section defines the exact subset of regular expressions implemented by the evaluator. A regular expression is interpreted as a pattern for the entire input string. It is not a substring-search engine: every input character must be consumed and the final automaton state must be accepting.

12.7.1 Supported operators and examples

Table 12.1: Supported regular-expression syntax.

Syntax	**Meaning**	**Example and interpretation**
Literal	A character matches itself.	`a` accepts only the one-character string `a`.
Concatenation	Expressions occur in sequence.	`abc` accepts the three-character string `abc`.
Union `|`	Either the left or right expression may match.	`a|b` accepts `a` or `b`.
Kleene star `*`	The preceding factor occurs zero or more times.	`a*` accepts the empty string, `a`, `aa`, and so on.
Parentheses	Group an expression and control precedence.	`(a|b)*` accepts any string made from `a` and `b`, including empty input.

12.7.2 Operator precedence and parser grammar

The parser's structure gives grouping and the postfix star the highest precedence, concatenation the next highest precedence, and union the lowest precedence. Concatenation is implicit: there is no concatenation symbol in the input.

A simplified grammar for the implemented syntax is:

$$\begin{aligned} \text{Expression} &\rightarrow \text{Term}\ (\texttt{|}\ \text{Term})^* \\ \text{Term} &\rightarrow \text{Factor}^+ \\ \text{Factor} &\rightarrow \text{Primary}\ (\texttt{*})? \\ \text{Primary} &\rightarrow \text{Literal} \mid \texttt{(}\ \text{Expression}\ \texttt{)} \end{aligned}$$

This grammar is a conceptual description of valid non-empty expressions. The implementation also has explicit checks for malformed forms such as a trailing union, a standalone star, an empty group, repeated stars, and unbalanced parentheses. Empty regular expressions are handled separately in `main.cpp`: the current program accepts an empty input when the regex is empty and rejects non-empty input for an empty regex.

Table 12.2: Parser functions and their responsibilities.

Function	Responsibility
`parse()`	Starts parsing, checks for a valid root node, and verifies that the entire expression was consumed.
`parseExpression()`	Parses a term and combines subsequent terms separated by `|` into `UnionNode` objects.
`parseTerm()`	Parses adjacent factors and combines them into `ConcatNode` objects.
`parseFactor()`	Parses a primary expression and optionally wraps it in a `StarNode`.
`parsePrimary()`	Parses a literal or a parenthesized expression; rejects invalid leading operators and empty groups.

12.7.3 Recursive-descent parser responsibilities

12.8 Detailed Object Model

12.8.1 Responsibilities of the main classes

The program is organized around a pipeline of classes, each with one main responsibility. This separation makes it easier to test and explain the implementation.

12.8.2 Expanded class relationship diagram

The following diagram complements the syntax-tree inheritance diagram in Figure 12.1. It shows the collaboration between the parser, automata builders, state objects, and simulators. Solid inheritance arrows in the syntax-tree diagram indicate an *is-a* relationship; the arrows below indicate use or construction relationships.

Table 12.3: Class responsibility reference.

Class	**Responsibility**
`RegexNode`	Abstract syntax-tree interface; declares the pure virtual `evaluate()` function and virtual destructor.
`LiteralNode`	Stores one literal character and returns its one-character string representation.
`ConcatNode`	Holds left and right child nodes and evaluates them in sequence.
`UnionNode`	Holds two alternatives and represents their union using `|`.
`StarNode`	Holds a child node and represents zero or more repetitions of that child.
`RegexParser`	Uses recursive-descent functions to validate and convert the input expression into a syntax tree.
`NFAState`	Stores a state identifier, final-state flag, and a list of symbol/destination transitions.
`NFAFragment`	Groups an NFA start state and end state so recursive construction can return a complete fragment.
`NFABuilder`	Recursively traverses the syntax tree and constructs NFA fragments using Thompson's construction.
`NFAOperations`	Provides epsilon-closure and move operations over sets of NFA states.
`NFASimulator`	Simulates an NFA by updating the reachable set of states for each input symbol.
`DFAState`	Represents a DFA state, the NFA-state subset associated with it, its final flag, and deterministic transitions.
`DFABuilder`	Converts the NFA to a DFA using subset construction and reuses existing subset states.
`DFASimulator`	Reads the input one character at a time, follows DFA transitions, and returns a Boolean result.

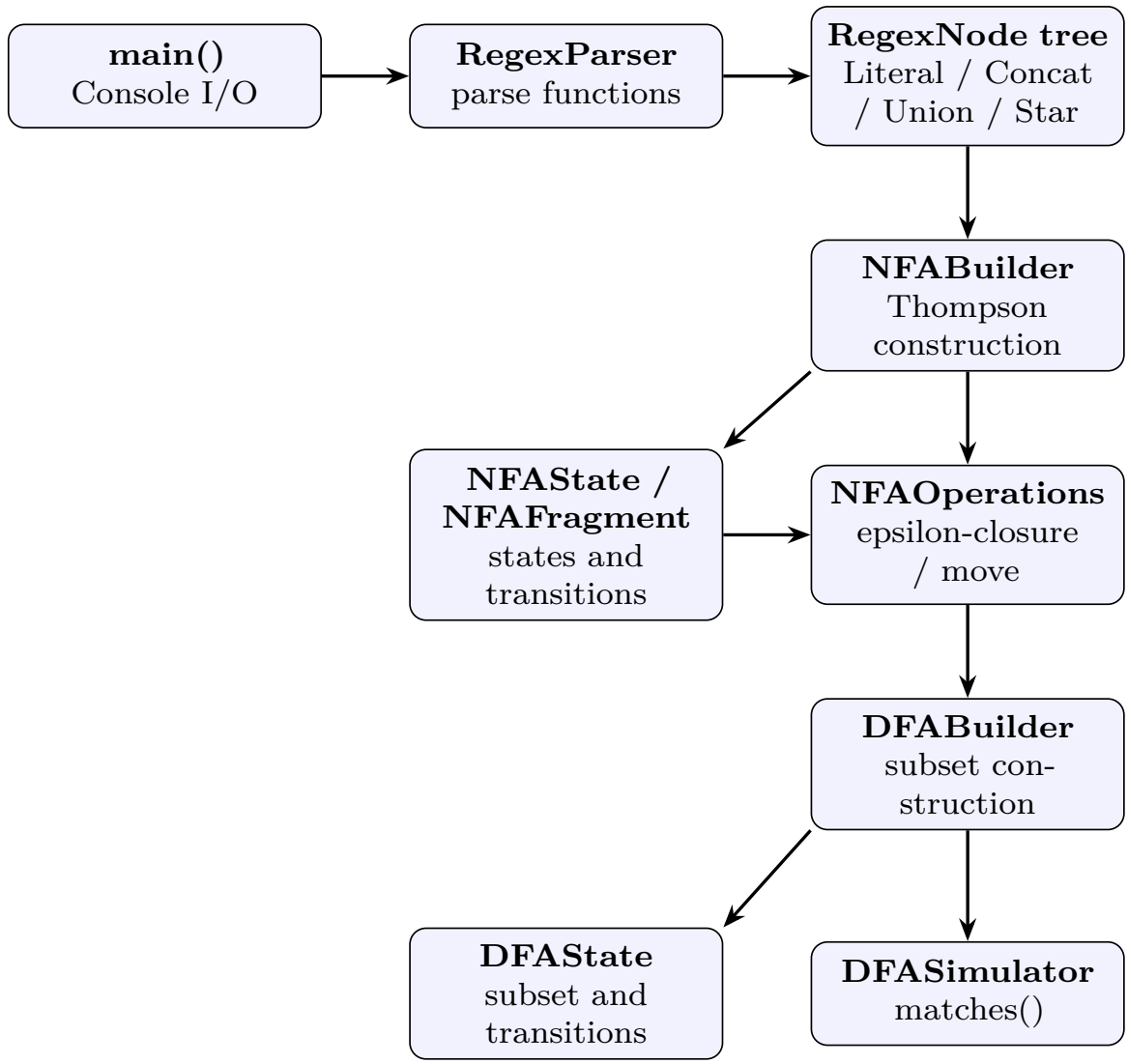


Figure 12.6: Collaboration diagram for the main implementation components.

12.8.3 Object-oriented design discussion

Abstraction. The abstract `RegexNode` class exposes the common `evaluate()` operation without specifying one representation for every expression.

Inheritance. `LiteralNode`, `ConcatNode`, `UnionNode`, and `StarNode` inherit from `RegexNode`. They can therefore be handled through the common base type.

Polymorphism. Each derived node overrides `evaluate()`. The NFA builder identifies the concrete node type using `dynamic_cast` and delegates construction to the matching builder function. This implementation uses runtime type checks in the builder; the virtual function is still part of the syntax-tree interface.

Encapsulation. The syntax-tree child pointers and literal values are private. Public getter functions expose the child nodes needed by the builder. State transitions are stored in state objects. Some state fields are public in the current implementation, so encapsulation is applied selectively rather than uniformly.

Composition and association. A concatenation node refers to two child nodes, a union node refers to two alternatives, and a star node refers to one child. An NFA fragment groups a start and end state. These are logical ownership/association relationships in the

current pointer-based design; the code does not use smart pointers to enforce ownership automatically.

12.9 Finite Automata: Visual Construction Walkthrough

This section uses small diagrams to explain the algorithmic ideas. The drawings are explanatory TikZ figures created in LaTeX, not screenshots of diagrams from an external editor.

12.9.1 NFA fragment for a literal

For a literal `a`, Thompson's construction creates a start state, one character transition, and a final state.

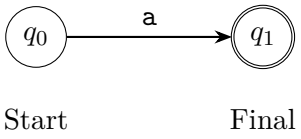


Figure 12.7: NFA fragment for the literal expression `a`.

12.9.2 NFA fragment for union

For `a|b`, a new start state has epsilon transitions to both literal fragments. Their accepting paths join at a new final state. The symbol `\epsilon` means the automaton can move without consuming an input character; in the C++ code this is represented by the null character `'\0'`.

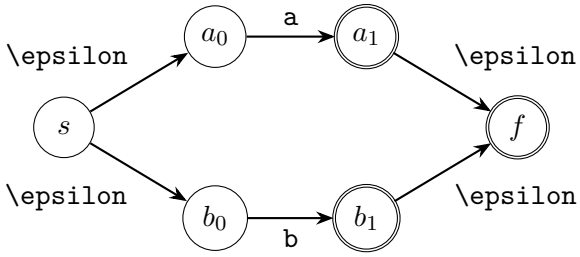


Figure 12.8: Conceptual Thompson NFA structure for union `a|b`.

12.9.3 NFA fragment for Kleene star

For `a*`, epsilon transitions permit zero repetitions by bypassing the child fragment, or one or more repetitions by returning from the child's accepting state to the child start. An additional epsilon path exits to the new final state.

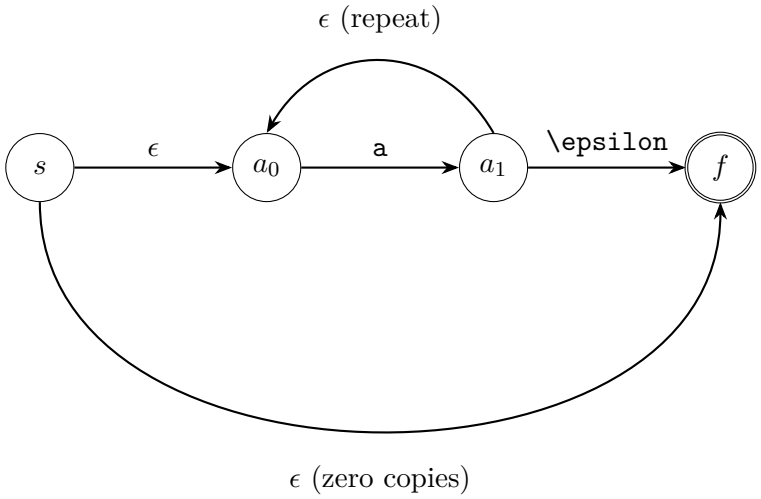


Figure 12.9: Conceptual Thompson NFA structure for `a*`.

12.9.4 How epsilon-closure and move work

Suppose the current reachable NFA state set is S. The epsilon-closure operation adds every state reachable from any state in S using zero or more epsilon transitions. The move operation follows only transitions labelled with a selected input symbol.

$$\text{move}(S, c) = \{q' \mid q \in S \text{ and } q \xrightarrow{c} q'\},$$
$$\epsilon\text{-closure}(S) = \{q' \mid q \text{ can reach } q' \text{ using only epsilon transitions}\}.$$

The NFA simulator first calculates the epsilon-closure of the start state. For each input character, it applies `move()` and then epsilon-closure again. The NFA accepts if the fragment's final state remains in the set after all input characters are consumed.

12.9.5 Subset construction: NFA states become DFA states

The subset construction creates a DFA state for each reachable set of NFA states. The set is the identity of that DFA state. When a newly calculated subset already exists, the builder reuses it instead of creating a duplicate. This is why the implementation keeps the represented NFA subset inside each `DFAState` object.

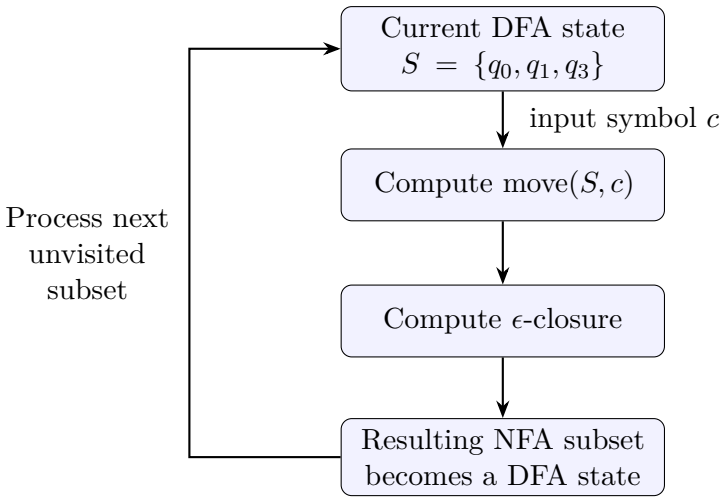


Figure 12.10: The central step in subset construction. The state names are illustrative.

12.10 Worked Example: Evaluating `(a|b)*abb`

The expression `(a|b)*abb` describes all strings over the alphabet {`a`,`b`} that end with `abb`. The prefix `(a|b)*` can match any sequence of `a` and `b`, including an empty prefix; the suffix `abb` must occur at the end.

12.10.1 Example input accepted

Consider the input `abababb`. It can be divided into the prefix `abab` and suffix `abb`. The prefix is accepted by `(a|b)*`, and the remaining three characters match the required suffix. Therefore, the expected result is `ACCEPT`.

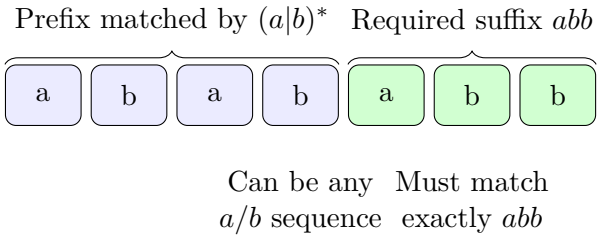


Figure 12.11: Visual decomposition of the accepted input `abababb`. Blue boxes represent the flexible prefix; green boxes represent the required suffix.

12.10.2 Example input rejected

The string `ab` is rejected because it ends before the required suffix `abb` has been completed. The string `abc` is also rejected: the character `c` is outside the alphabet described by this expression, and the required suffix is not matched. These examples demonstrate that a valid prefix alone is insufficient; the full input must reach a final DFA state.

12.10.3 DFA simulation trace concept

The DFA simulator stores a pointer to the current state. For every character, it looks up the matching transition. If the transition does not exist, the method returns `false`. If all characters are processed, the value of `isFinal` on the last state determines acceptance. The implementation does not display the internal state trace in the console; the following table is a conceptual trace of the simulator's decision process.

Table 12.4: Conceptual decision trace for the accepted sample.

Stage	Action	Reason
Initial	Start at DFA start state.	No input has been consumed yet.
For each character	Follow the transition labelled by that character.	DFA state and input symbol determine at most one next state.
All characters consumed	Inspect the current state's final flag.	Acceptance depends on the complete input, not on a partial match.
Final check	Print `ACCEPT` if final; otherwise print `REJECT`.	This is the Boolean result returned by `DFASimulator::matches()`.

12.11 Algorithm Pseudocode

Pseudocode below summarizes the major algorithms without tying the explanation to C++ syntax.

12.11.1 Recursive-descent parsing

```
parse():
    if expression is empty: return invalid/root-null
    root = parseExpression()
    if root is invalid: return invalid/root-null
    if not all characters were consumed: return invalid/root-null
    return root

parseExpression():
    node = parseTerm()
    while next character is '|':
        consume '|'
        right = parseTerm()
        if right is invalid: return invalid/root-null
        node = UnionNode(node, right)
    return node
```

12.11.2 Thompson construction

```
build(node):
    if node is LiteralNode:
        create start and final states; connect by literal
    if node is ConcatNode:
        left = build(node.left); right = build(node.right)
        connect left.final to right.start by epsilon
        return (left.start, right.final)
    if node is UnionNode:
        build both alternatives
        create new start and final states
        epsilon-branch to both alternatives and rejoin
    if node is StarNode:
        build child fragment
        create new start and final states
        add epsilon paths for zero copies, repetition, and exit
```

12.11.3 DFA matching

```
matches(start, input):
    current = start
    for each character c in input:
        if current has no transition on c:
            return false
        current = transition(current, c)
    return current.isFinal
```

12.12 Complexity and Resource Considerations

Let m be the length of the regular expression and n be the length of the input string. Parsing is typically linear in the expression length for this grammar, excluding the costs of allocation and error cleanup. Thompson construction creates a number of NFA states proportional to the syntax-tree size, so its state count is $O(m)$ for this supported syntax.

NFA simulation can maintain a set of reachable states for each character. If the NFA has s states and transitions are scanned directly, a straightforward upper-bound description is $O(n(s+t))$, where t accounts for transition scans and closure traversal per character; exact runtime depends on the transition graph and epsilon closures. Subset construction can produce up to 2^s DFA states in the worst case because every DFA state represents a subset of NFA states. DFA simulation is $O(n)$ when transition lookup is treated as constant-time on the map implementation for a small alphabet; in this code, each lookup uses a C++ map and therefore has logarithmic lookup cost in the number of outgoing transitions.

The implementation uses raw pointers for syntax-tree nodes and automaton states. This is simple for a teaching implementation but does not automatically release all allocated objects. A production-quality version should introduce clear ownership, preferably with smart pointers or dedicated owner containers, and should clean up partially built structures

when parsing fails. Memory-management improvements are future work and should not be mixed into the final documentation-only changes unless they are tested separately.

12.13 Complete Test Coverage Summary

The project's PowerShell test runner compiles the source files and executes 21 cases. The development run previously reported 21 passed and 0 failed. The table below summarizes the categories covered by the script; run `.\tests.ps1` again before final submission so the recorded result matches the final files.

Table 12.5: Test categories covered by the automated suite.

Category	**Representative checks**
Literal	Match and mismatch for `a`.
Concatenation	Match the sequence `abc`.
Union	Accept either branch and reject a non-branch character.
Kleene star	Repeated characters, empty input, and mismatching characters.
Parentheses	Single grouping and nested expressions.
Combined operators	Complex expression `(a|b)*abb` with accepted and rejected inputs.
Empty cases	Empty regex with empty input and with non-empty input.
Malformed expressions	Trailing union, standalone star, repeated star, missing/extra parenthesis, and empty group.

12.14 Diagram-Based Demonstration Guide

All UML and algorithm diagrams in this report are generated directly in LaTeX using the TikZ package, so they remain editable in the source file. The console interaction shown in the implementation results section is a genuine screenshot of the program running in PowerShell.

For a clear project demonstration, follow the sequence below and refer to the corresponding TikZ figures in this report:

1. Enter `(a|b)*abb` as the regular expression and `abababb` as the input. Explain that the expression allows any sequence of `a` and `b`, followed by the suffix `abb`.

2. Show the architecture diagram and trace the stages from console input through parsing, NFA construction, DFA construction, and simulation.

3. Use the class diagrams to explain the abstract `RegexNode` interface and its derived syntax-tree node classes.

4. Use the literal, union, and Kleene-star NFA figures to explain labelled transitions and epsilon transitions.

5. Explain epsilon-closure and the subset-construction figure: one DFA state represents a set of NFA states.

6. Compare the accepted input `abababb` with the rejected input `ab`. The latter does not finish with the required suffix `abb`.

7. Finish with the test coverage table and state the supported syntax and current limitations.

12.14.1 TikZ Result-State Illustration

The following small diagram provides a report-native visual summary of the final decision. It is intentionally a static explanatory figure; the actual decision is produced by the C++ simulator at runtime.

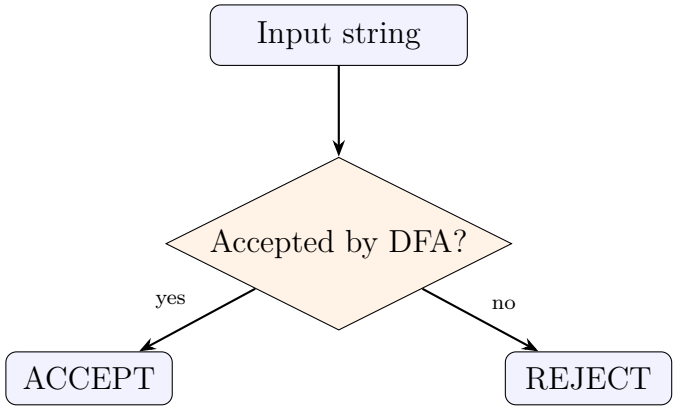


Figure 12.12: TikZ illustration of the final DFA acceptance decision.

12.15 Implementation and Usage

The project is implemented in C++ and uses the Standard Template Library, including `std::string`, `std::vector`, and `std::set`. The code is separated into header files in `include/` and implementation files in `src/`. The PowerShell test runner is located in the project root as `tests.ps1`.

Compile from the project root with a C++ compiler such as g++:

```
g++ -Iinclude src/*.cpp -o src/a.exe
```

Run the program on Windows using:

```
.\src\a.exe
```

The program asks for a regular expression and then an input string. For automated testing, run the following command in PowerShell from the project root:

```
.\tests.ps1
```

12.16 Results and Testing

The project includes a test script with 21 cases covering literals, concatenation, union, Kleene star, parentheses, nested expressions, a complex expression, empty input, and malformed expressions. The previously reported test run completed with all 21 cases passing and no failures. This is the recorded result from the development test run; rerun the script against the final submitted files to reconfirm it.

Table 12.6: Representative functional test cases.

Regular Expression	Input String	Expected Result
`a`	`a`	ACCEPT
`a`	`b`	REJECT
`a|b`	`b`	ACCEPT
`a*`	`aaa`	ACCEPT
`(a|b)*abb`	`abababb`	ACCEPT
`(a|b)*abb`	`ab`	REJECT
`(a|b)*abb`	`abc`	REJECT
`a|`	`a`	REJECT
`()`	empty string	REJECT

```
PS D:\RegexProject> g++ -Iinclude src/*.cpp -o src/a.exe
PS D:\RegexProject> src\a.exe
Enter Regular Expression: (a|b)*abb
Enter String: abababb
Result: ACCEPT
```

Figure 12.13: Console Interaction of the Regex Evaluator

12.17 Limitations and Future Enhancements

The current evaluator supports literals, concatenation, union, Kleene star, and parentheses. It does not currently implement advanced regex syntax such as character classes, escape sequences, optional matching, one-or-more repetition, or wildcard matching. The implementation also uses raw pointers for syntax-tree and automaton objects; smart pointers and explicit ownership management would improve memory safety. Possible future improvements include additional operators, clearer diagnostics for malformed patterns, memory-management improvements, and a visual display of NFA/DFA states and transitions.

12.18 Conclusion

The Regex Evaluator demonstrates the stages involved in implementing a basic regular-expression engine without relying on a built-in regex library. Recursive-descent parsing builds a syntax tree, Thompson's construction converts the tree into an NFA, subset construction produces a DFA, and DFA simulation checks the input string. The project applies abstraction, inheritance, polymorphism, encapsulation, and STL containers in

a modular C++ implementation. The automated tests provide repeatable checks for supported operators and common invalid expressions.